

Robotics and Autonomous Systems

Robotic Mirror Hand

Marcelo Ruiz Leal, Bogdan-Cristian Ivanescu,
Tatiana Vysinska, Konstantinos Kanelatos

Table of Contents

1. Introduction.....	3
2. Hardware.....	3
2.1 Robotic Hand Components.....	3
2.2 Glove Components.....	5
3. System Design.....	7
3.1 1st Stage Hardware Architecture.....	7
3.2 2nd Stage Hardware Architecture.....	9
3.3 Final Glove-Controlled Architecture.....	11
4. Software.....	14
5. Testing and Results.....	16
Test 1: Individual Finger Movement.....	16
Test 2: Power Supply Testing.....	16
Test 3: PCA9685 Controller Module Testing.....	17
Test 4: NRF24L01 Wireless Communication Testing with the Glove System.....	18
Test 5: Glove-Controlled Robotic Hand Operation.....	18
6. Conclusion.....	19
References.....	21

1. Introduction

This project was originally started by a previous student team at the University of Applied Sciences. As part of the Robotics and Autonomous Systems course, our group was assigned to continue the development of a 3D-printed robotic hand with movable fingers.

The main goal of our work was to improve the existing system by repairing the wiring, upgrading the power supply, and making the robotic hand more reliable overall. During the development process, we examined the existing design, identified a number of issues, and implemented a number of improvements to both the hardware and the control system.

Another important objective was the development of a wired glove equipped with flex sensors. The purpose of the glove was to allow the robotic hand to copy the movements of a user's hand in real time. To achieve this, several hardware and software components had to be integrated and tested.

This report describes the components used in the project, the design of the system, and the software developed for controlling the robotic hand. It also presents the different stages of development, the modifications made to the original design, and the testing procedures that were carried out throughout the project.

2. Hardware

2.1 Robotic Hand Components

Components	Model	Quantity	Purpose
Microcontroller	ESP32 38-pin	1	Receiving controller; listens for glove data and controls the servos
Servo Motors	TowerPro SG-5010	5	Finger actuation (pulling/flexing fingers)
Power Supply	5V 3A Wall Adapter Plug	1	External power distribution for the servo motors
Servo Board	PCA9685 16-Channel PWM	1	Drives all 5 servo motors using only 2 I2C pins from the receiving ESP32

Table 1.1 Robotic Hand Components

Microcontroller The original microcontroller used in the system was Arduino UNO, but after further evaluation, the decision was made to switch to the ESP32. The primary reason for choosing the ESP over the original Arduino UNO was due to its built-in Bluetooth capabilities, which allowed for the complete removal of the external NRF transmitter and controller modules. An ESP32 was used on both the robotic hand (receiver) and the glove (transmitter) as the central processing units (JLCPCB, 2026).

Servo Motors Servo motors were used for finger actuation, with one dedicated motor per finger. Actuation is achieved by connecting the string, which is going through the inside of the 3D-printed finger, to the plastic attachment on the top of the motor. When the servo rotates, the string tightens or loosens according to the motion, causing the finger to bend or rest.

The specific model of the servo motors is TowerPro SG-5010. It is designed to safely operate between 4.8V and 6V. The table below compares the different capabilities of the servo operating under different voltages (Mahcene, n.d.).

Voltage	Stall Torque	Operating Speed (per 60°)
5V (Current Setup)	5.5 kg . cm	0.20 s
6V (Maximum Limit)	6.5 kg . cm	0.16 s

Table 1.2 Servo Motor under Different Voltages

While providing a higher voltage would increase the speed and torque of the fingers, a 5V was decided on, mainly due to simplicity and sufficiency.

Power Supply The initial power supply consisted of a standard 5V, 1A wall adapter, with the end stripped and separated into the two separate cables, positive and negative, directly plugged into the breadboard. However, this configuration provided insufficient current. To get all the servos to move, especially at the same time, 1A current powering all five was simply not enough.

To resolve this, a new power supply was implemented consisting of two parts:

Power Adapter A wall adapter capable of delivering 5V and up to 3A of current. The increased current overhead allows multiple servos to operate simultaneously with minimal

delay and no motor jittering.

DC Barrel Jack Adapter A female barrel jack adapter was connected to the end of the power supply. Two jumper wires were secured into its screw terminals, allowing the 5V and Ground rails to be routed directly into the servo controller board.

Servo Board - PCA9685 board This component was added to ensure a smoother connection between the power supply and servos. Servos are directly connected to the PCA9685 board, which splits the power and current from the power supply evenly among all the servos connected to it. The PCA9685 board works as the power strip and translator in one and allows 5V and 5A current to be directly transmitted in the servos without the risk of damaging them or other equipment (Adafruit Industries, 2026). A standard breadboard is not designed to handle more than 1A to 1.5A of continuous current going through it without overheating or causing voltage drops, hence using a servo board is a better alternative.

Receiver / Transmitter In the original system, an external NRF chip was used to wirelessly transfer values between the robotic hand and the computer. However, by upgrading to the ESP32, those external chips were completely eliminated, since the system now utilizes the ESP32's built-in wireless capabilities.

2.2 Glove Components

Component	Model	Quantity	Purpose
Microcontroller	ESP32 38-pin	1	Main controller, Bluetooth transmitter
Finger Sensors	MPU-6050 (GY-521)	5	Finger movement detection
I2C Multiplexer	TCA9548A	1	Allows 5 sensors on same I2C bus
Battery	EEMB 3.7V 1100mAh LiPo	1	Wireless power supply
Charging Module	TP4056 with protection	1	Battery charging and over-discharge protection
Breadboard	Mini breadboard	1	Shared 3.3V and GND power bus

Table 1.3 Glove Components

The smart glove reads finger movements using 5 MPU-6050 accelerometers, one per finger, connected through a TCA9548A I2C multiplexer to an ESP32 microcontroller. The system detects 3 finger states per finger (straight, half closed, fully closed) with the exception of the thumb which only has 2 states (straight and fully closed). The glove communicates wirelessly with a robotic hand via Bluetooth.

Finger Sensors The MPU-6050 (GY-521) is a 6-axis motion tracking sensor used to capture the physical movement of the user's fingers so it can be replicated by the robotic hand. It combines a 3-axis gyroscope with 3-axis accelerometer, allowing it to simultaneously measure linear acceleration and angular velocity across three spatial axes (X, Y and Z), providing the precise positional and tilt data required to calculate finger movement.

I2C Multiplexer The TCS9548A multiplexer acts as a multi-way switch for electronics. Because all five finger sensors share the same IC2 address, connecting them directly to the same pins on the microcontroller would cause a data collision. The multiplexer resolves this by allowing the microcontroller to go through eight independent I2C channels, communicating with each identical sensor one at a time without any interference.

Battery To ensure the glove is portable and can be comfortably worn, a wireless power supply was necessary. Instead of relying on a power cable, the glove is powered with a EEMB 3.7V 1100mAh LiPo battery. This power source provides the necessary capacity to operate the ESP32 and the sensor array wirelessly.

Charging Module A TP4056 charging module with built-in protection was added to safely manage the LiPo battery. If a battery would be used without this charging module, it would risk overcharging or draining the power too low.

Breadboard A small breadboard was used to create connections between the components without having to solder anything. Since the glove design required a lot of testing and adjustments, using a tiny breadboard was the perfect solution to connect everything, while still being able to make changes if required.

3. System Design

3.1 1st Stage Hardware Architecture

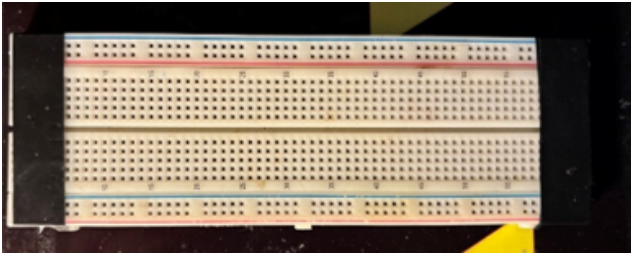
One 3D printed hand 	Five motor sensors 
One breadboard 	One Arduino uno device 
One adaptor 	The power plug (power supply) 

Table 2.1 1st Stage Hardware Architecture

The robotic hand is controlled by five servo motors, with one servo motor assigned to each finger. Each finger has a string going through it, with an end tied to the head of the servo

motor, making the movement of the finger done by tugging / releasing the set string. This one-to-one mapping allows each finger to move independently.

Thumb Flex Sensor → Thumb Servo Motor

Index Flex Sensor → Index Servo Motor

Middle Flex Sensor → Middle Servo Motor

Ring Flex Sensor → Ring Servo Motor

Pinky Flex Sensor → Little Finger Servo Motor

Each servo motor is connected using three wires:

- A power wire connected to the breadboard.
- A ground wire (GND) connected to the breadboard.
- A signal wire connected to a digital pin (A1-A5) of the Arduino Uno.

The breadboard distributes power and ground to all five servo motors. The Arduino Uno is connected to a laptop via a USB cable. Through the Arduino, the uploaded program generates the control signals that determine the position of each servo motor (more detailed later to the software part). When a control signal is sent from the Arduino, the corresponding servo motor rotates and pulls a string attached to the finger. This action bends the finger of the robotic hand.

Since each finger has its own servo motor, the fingers can move independently and accurately reproduce programmed movements, allowing the robotic hand to mimic the movements of a human hand.

The Arduino Uno shares a common ground connection with the breadboard to ensure proper circuit operation. Power for the entire system is supplied through a power adapter, from the power plug, connected to the breadboard, which provides the necessary current for the servo motors and the robotic hand mechanism.

System Flow

Laptop → Arduino Uno → Servo Motors → String Mechanism → Robotic Hand Movement

3.2 2nd Stage Hardware Architecture

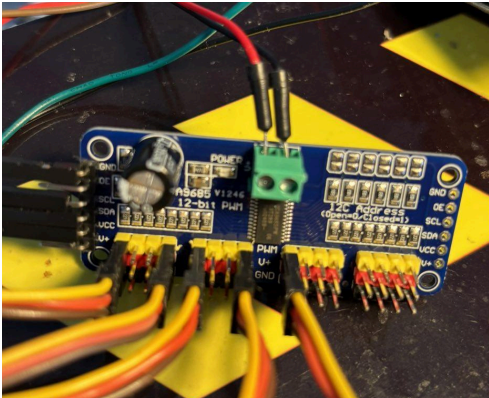
One 3D printed hand 	Five motor sensors 
PCA9685 Servo Driver Module 	One Arduino uno device 
One adaptor 	The power plug (power supply) 

Table 2.2 2nd Stage Hardware Architecture

The second design of the robotic hand does not change a lot. There are again five servo motors, one assigned to each finger, allowing independent finger movement and accurate reproduction of human hand gestures.

The control architecture is based on a PCA9685 Servo Driver Module. The PCA9685 module was integrated into the system as a dedicated 16-channel PWM controller. The Arduino Uno communicates with the PCA9685 servo controller through the I2C protocol. The communication uses two dedicated lines: SDA (Serial Data) for data transfer and SCL (Serial Clock) for synchronization. Through this interface, the Arduino transmits the desired servo positions, while the PCA9685 generates the corresponding PWM signals for the servomotors.

The mapping between the flex sensors and the servo motors remains unchanged:

- Thumb Flex Sensor → Thumb Servo Motor
- Index Flex Sensor → Index Servo Motor
- Middle Flex Sensor → Middle Servo Motor
- Ring Flex Sensor → Ring Servo Motor
- Pinky Flex Sensor → Little Finger Servo Motor

The Arduino Uno continuously reads the values from the five flex sensors and converts them into movement commands. These commands are transmitted to the PCA9685 module. The PCA9685 then generates the appropriate PWM signals for each servo motor independently.

Each servo motor is connected to one of the PCA9685 output channels through:

- Power (VCC)
- Ground (GND)
- PWM Signal

The servo motors are mechanically linked to the fingers of the robotic hand through a string-based actuation mechanism. When a PWM signal is received, the corresponding servo motor rotates and pulls the attached string, causing the finger to bend. This arrangement allows the robotic hand to mimic the movements of the user's hand with high accuracy.

A common ground connection is shared between the Arduino Uno, the PCA9685 module, and the power supply to ensure reliable communication and stable operation. Power for the servo motors is provided through an external power source, the same as in the second stage, connected directly to the PCA9685 module, which efficiently distributes the required current to all servomotors.

System Flow

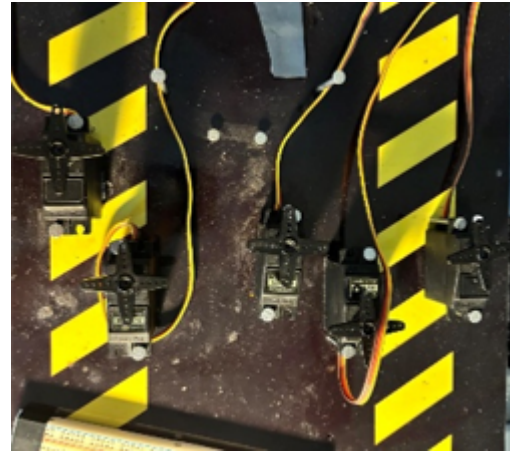
Laptop → Arduino Uno → PCA9685 Servo Drive → PWM Signals → 5 Servo Motors → Robotic Hand Fingers

3.3 Final Glove-Controlled Architecture

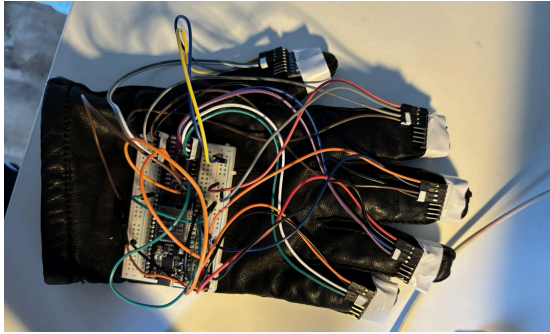
One 3D printed hand



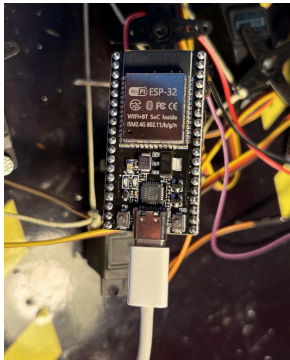
Five motor sensors



One glove with breadboard, Microcontroller(ESP-32), finger flex sensors and an I2C multiplexer on the glove:



Another Microcontroller(ESP-32S)



Power bank



The power plug (power supply)



One adaptor



PCA9685 Servo Driver Module

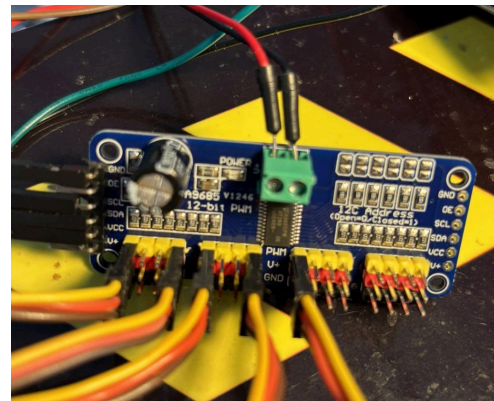


Table 2.3 Final Glove-Controlled Architecture

The final design of the robotic hand introduced a glove-based control system and replaced the Arduino Uno with a dual-microcontroller architecture. The robotic hand still uses five servo motors, one assigned to each finger, allowing independent finger movement and accurate reproduction of human hand gestures.

The glove is equipped with five finger flex sensors, one attached to each finger. These sensors detect the bending of the user's fingers by changing their resistance values. A microcontroller mounted directly on the glove continuously reads the sensor values and processes the movement data. To provide portability and independent operation, the glove electronics are powered by a power bank.

Communication between the glove and the robotic hand is achieved through the I2C protocol. An I2C multiplexer is used to manage communication between the devices and ensure reliable data transfer. The processed sensor data are transmitted from the glove microcontroller to a second microcontroller located on the robotic hand side.

The second microcontroller communicates with the PCA9685 Servo Driver Module through the I2C interface. The PCA9685 acts as a dedicated 16-channel PWM controller and generates the PWM signals required to drive the servomotors.

The mapping between the flex sensors and the servo motors remains unchanged:

- Thumb Flex Sensor → Thumb Servo Motor
- Index Flex Sensor → Index Servo Motor
- Middle Flex Sensor → Middle Servo Motor
- Ring Flex Sensor → Ring Servo Motor
- Pinky Flex Sensor → Little Finger Servo Motor

Each servo motor is connected to one of the PCA9685 output channels through:

- Power (VCC)
- Ground (GND)
- PWM Signal

The servo motors are mechanically linked to the fingers of the robotic hand through a string-based actuation mechanism. When the user bends a finger, the corresponding flex sensor detects the movement and sends the information to the glove microcontroller. The data is then transmitted through the I2C communication network to the second microcontroller and finally to the PCA9685 module. The PCA9685 generates the appropriate PWM signal, causing the corresponding servo motor to rotate and pull the attached string. This action bends the robotic finger and reproduces the user's movement in real time.

A common ground connection is shared between the control electronics and the servo control system to ensure stable communication and reliable operation. Power for the servomotors is provided by an external power source connected to the PCA9685 module, while the glove electronics are powered independently through a portable power bank.

System Flow

User Hand → Flex Sensors (Glove) → Microcontroller (Glove) → I2C Multiplexer → Microcontroller (Robotic Hand) → PCA9685 Servo Driver → PWM Signals → 5 Servo Motors → Robotic Hand Fingers

This architecture was selected as the final implementation of the project because it provides improved portability, reliable communication, and real-time gesture control of the robotic hand.

4. Software

As stated previously, our aim for this project was to mimic the movement of the glove worn on a real hand on the robotic hand model. Since it would be quite complicated to imitate any movement made by the hand, we decided to create 3 unique states for each finger, except the thumb, which due to the model limitation, is not able to bend as much as the other fingers.

Each finger is classified into discrete states based on the accelerometer X-axis reading. The palm must be facing downward for accurate state detection. Calibration was performed with the hand in a natural position.

State	Angle	Description	Thumb
0	0°	Straight / fully open	Open
1	45°	Half closed	Not used
2	90°	Fully closed	Closed

Table 3.1 State Definitions

These thresholds were measured experimentally with the glove worn on the right hand, palm facing down.

Finger	Channel	State 0 (Straight)	State 1 (Half)	State 2 (Closed)
Pinky	CH0	$X > -8000$	-8000 to -13000	$X < -13000$

Ring	CH1	X > -5500	-5500 to -12000	X < -12000
Middle	CH2	X > -5500	-5500 to -11500	X < -11500
Index	CH3	X > -10500	-10500 to -14000	X < -14000
Thumb	CH4	X > -6200	—	X < -6200

Table 3.2 Calibration Thresholds (X-axis)

After the finger sensors detect the current state of each finger, it stores set values in an array, e.g. [0, 2, 1, 2, 1], it then transmits the set array to the hand by the ESP32 microcontroller via Bluetooth - one connected to the glove, the other on the hand. After the ESP32 microcontroller the robotic hand receives these values, it then gives a signal to each servo to move a set amount.

The servos are controlled by PMW, ranging from 150 to 500. In order to achieve the middle state, we needed to set the values for them, which was done experimentally, since the motion and flexibility of each finger varied.

```
const int servoMid[5] = {400, 450, 450, 400, 250};
```

The other two states - maximum and minimum were just the highest and lowest values possible. Following this, a code was created to move each servo according to the value received (0, 1, 2).

```
void setFinger(int portIndex, int state) {
    int pulse;
    if (portIndex == 0) { // Thumb - 2 states
        pulse = (state == 2) ? SERVOMAX : SERVOMIN;
    } else {
        if (state == 0) pulse = SERVOMIN;
        else if (state == 1) pulse = servoMid[portIndex];
        else pulse = SERVOMAX;
    }
    pwm.setPWM(servoPorts[portIndex], 0, pulse);
}
```

When executed, each servo performs the set rotation according to their values,

tugging on the string attached inside the finger, mimicking the state of the glove on the real hand.

5. Testing and Results

Test 1: Individual Finger Movement

Each servo motor was tested separately to verify that it could correctly move the corresponding finger of the robotic hand.

Results/Problems:

- All five servo motors responded to the commands sent by the Arduino Uno.
- Each finger was able to bend and return to its initial position.
- The one-to-one mapping between servo motors and fingers worked as expected.

Test 2: Power Supply Testing

One of the main challenges during the development of the robotic hand was providing sufficient power to all five servo motors.

The first power supply used in the project had a maximum output current of 1A. Since each servo motor can require up to 1A under load, the available current was not sufficient to operate all servos reliably at the same time.

Results/Problems:

- Some servo motors did not respond correctly.
- The robotic hand could not perform simultaneous finger movements.
- A possible cause was the limited current output of the power supply.
- Additionally, the power supply cables may have been partially damaged, further affecting performance.

Since it was yet unclear whether the issue was the original power supply or the malfunction of the servo motors, the next test involved powering the servo motors individually directly through the Arduino Uno, which received power from the laptop via USB.

Results/Problems:

- The system was able to operate a single servo motor successfully.
- However, the Arduino could not provide enough current for all five servo motors simultaneously.
- To test the motors, each servo had to be connected and tested individually.

The final solution was to use a 3A power supply connected to the breadboard. The breadboard distributed power and ground to all five servo motors simultaneously.

Results/Problems:

- All five servo motors operated correctly.
- The system became significantly more stable and reliable.
- Although all five servo motors were successfully powered at the same time, simultaneous movement of all fingers was not tested during this stage of the project.

Test 3: PCA9685 Controller Module Testing

The purpose of this test was to verify the correct operation of the PCA9685 servo driver board and establish stable communication with the microcontroller using the I2C protocol.

To manage all five finger servos without overwhelming the microcontroller's limited pin count, the PCA9685 module was introduced to act as a dedicated multi-channel PWM signal generator. This test focused on ensuring that control data could be sent smoothly over the I2C bus and that the board could distribute signals to multiple motors simultaneously.

Results/Problems:

- The control code was loaded and executed successfully without any compilation or register assignment errors.
- The PCA9685 correctly generated the required PWM signals, allowing the motors to execute programmed movements and angles fluidly.
- A multi-channel load test was successful, allowing five servo motors to be connected, powered, and driven at the same time to simulate individual finger control.

- The module demonstrated efficient current management, maintaining electrical and systemic stability throughout the entire sequence of movements.

The test was completely successful. The PCA9685 module proved capable of providing the necessary channels and signal stability required for the mechanism, and was validated to be permanently integrated into the robotic hand system.

Test 4: NRF24L01 Wireless Communication Testing with the Glove System

The purpose of this test was to establish wireless communication between the glove and the robotic hand using NRF24L01 transceiver modules connected to Arduino-based controllers. One Arduino board was connected to the glove unit and acted as the transmitter, while a second Arduino board was connected to the robotic hand and acted as the receiver.

Results/Problems:

- The NRF24L01 modules were configured and connected to the Arduino boards.
- Several communication tests were performed to transmit sensor data from the glove to the robotic hand.
- As a result, the robotic hand could not consistently reproduce the movements detected by the glove.

The communication test with the Arduino and NRF24L01 modules did not produce the expected results. Since the connection was not stable enough for reliable operation of the robotic hand, we decided not to continue with this setup and instead moved to a different microcontroller-based solution, which was tested in the next stage of the project.

Test 5: Glove-Controlled Robotic Hand Operation

The purpose of this test was to verify the operation of the robotic hand using the glove-based control system. In the final configuration, two microcontrollers were used: one mounted on the glove to acquire sensor data and a second microcontroller connected to the PCA9685 module to control the servo motors. The robotic hand movements were generated

from the signals produced by the glove sensors, allowing the robotic fingers to follow the movements performed by the user in real time.

Results/Problems:

- The robotic hand successfully responded to the movements performed by the user wearing the glove.
- Finger bending motions detected by the glove were transmitted to the robotic hand and reproduced in real time.
- An issue was observed when the pins of the microcontroller connected to the PCA9685 module were accidentally touched. Unintended signals were occasionally generated, causing the robotic fingers to move on their own without any input from the glove.
- Another limitation observed during testing was related to the orientation of the glove. The robotic hand responded correctly when the user's palm was facing downward. However, when the palm was facing upward, sideways, or held vertically, the system did not respond consistently and little or no movement was detected.

The test was considered successful, as the robotic hand correctly followed the user's hand movements through the glove interface. Despite minor operational limitations, the overall performance demonstrated the feasibility and effectiveness of the proposed control system.

6. Conclusion

This project focused on continuing and improving an existing 3D-printed robotic hand developed by a previous student team. During the project, the hardware and control system were examined, tested, and modified to improve the overall performance and reliability of the robotic hand.

The main improvements made to the system were:

- Repair and replacement of damaged wiring.
- Improvements to the power supply system to ensure stable operation of the servomotors.
- Integration of a PCA9685 servo controller for more efficient control of multiple

servomotors.

- Testing and validation of the different hardware configurations throughout the development process.

In the final stage of the project, a glove-based control system was added, allowing the robotic hand to follow the user's finger movements in real time. After several tests and adjustments, the final setup operated reliably and achieved the main goal of the project: controlling the robotic hand through a wearable glove interface.

Although the project was successful, there are a few things, which can be improved in the future. One of the major changes is the orientation of the glove on the hand. Currently, the glove reacts to the movement, however only when the palm is facing down. It does not cause any issues like this, but for a better visual effect, making it work in the same orientation as its hand, would be more favourable.

Another change would be to improve the visual look of the wiring. Cleaning up the cables or figuring out a way to minimize the overall cables used would not only give the whole system a more clean and organized look, but also make it slightly easier to handle.

References

JLCPCB. (2026). ESP32 vs Arduino: Which is better for your project? JLCPCB Tech Blog.
<https://jlcpcb.com/blog/esp32-vs-arduino>

Mahcene, I. (n.d.). TowerPro SG-5010 servo overview. Scribd.
<https://www.scribd.com/document/522821543/SG-50-TowerPro>

Adafruit Industries. (2026). *Adafruit PCA9685 16-channel 12-bit PWM/servo driver overview*. Adafruit Learning System.
<https://learn.adafruit.com/16-channel-pwm-servo-driver/overview>